GRADO EN CIENCIA E INGENIERÍA DE DATOS

Proyecto

Infraestructura para la Computación de Altas Prestaciones

Curso: 2024/2025

Ángel González de Santos

Rubén Del Valle Carrillo

ÍNDICE

Tarea 1 3

Tarea 2 7

Tarea 3 16

Anexo de automatización 23

Resumen de recursos y servicios desplegados 34

Tarea 1

Primero, se debe crear una **tabla Dynamo DB** con clave de partición fecha y nombre 'proy-tabla'. Esta tabla tendrá cuatro columnas distintas, 'fecha' (que usaremos con indexación para que se diferencien los distintos valores de la tabla), 'mes_anyo' (para poder hacer una diferenciación mensual necesaria para posteriores apartados de la práctica), 'desviacion' y 'media'.

Hemos elegido 'fecha' clave de partición ya que necesitamos un valor que nos ayude a buscar días suelto que nos ayudan a los cálculos necesarios para la siguiente tarea. Al principio pensamos en utilizar la variable 'mes_anyo' como clave de partición, pero debido a que nos dificulta a la hora de encontrar valores concretos preferimos dejarlo así.

Ahora creamos el **bucket S3** donde añadiremos el archivo de datos csv original. Lo llamaremos 'proy-ingestadatos'.

A continuación creamos la **notificación SNS**, la llamaremos 'proy-alarma-temperatura' y será del tipo Estándar. Le deberemos añadir una suscripción de protocolo email añadida a nuestro correo para comprobar que las notificaciones llegan correctamente.

Finalmente debemos crear las **dos funciones lambda** que manejarán nuestros datos y nos notificarán de ello. La primera de ella se llamará 'proy-funcion-procesador-datos' con Python 3.9, además deberemos cambiar el rol de ejecución a LabRole. Deberemos agregar como desencadenador el bucket s3 creado anteriormente para que pueda coger los datos de ahí cuando carguemos el archivo csv. También en configuración cambiaremos el tiempo de espera a 1 minuto para que le diese tiempo a seleccionar todos los datos ya que la función datosTable.put_Item() tarda demasiado tiempo con la cantidad de elementos que tenemos por lo que había que aumentar ese tiempo.

Esta función deberá seleccionar los valores brutos que encontramos en el archivo csv y limpiarlos para añadirlos a la tabla DynamoDB y además crear la variable 'mes_anyo' a partir de los valores de fecha.

Arquitectura:

```
# 1. Obtener detalles del archivo subido desde el evento S3
bucket_name = event['Records'][0]['s3']['bucket']['name']
file_key = urllib.parse.unquote_plus(event['Records'][0]['s3']['object']['key'])

print(f"Procesando archivo: {file_key} del bucket: {bucket_name}")
```

- Obtenemos el nombre del bucket y la key para poder recoger la información del bucket S3 al que mandamos el archivo csv

```
# 2. Leer el archivo CSV desde S3
response = s3.get_object(Bucket=bucket_name, Key=file_key)
content = response['Body'].read().decode('utf-8')

# 3. Parsear el CSV
csv_reader = csv.DictReader(io.StringIO(content), delimiter=',') # Ajusta delimitador si es necesario
```

- Entramos al bucket S3 y leemos el archivo csv

```
for row in csv_reader:
    # Formato esperado del CSV: Fecha, Medias, Desviaciones
    fecha = row['Fecha'] # Ej: 22/03/2017
    media = row['Medias']
    desviacion = row['Desviaciones']


    partes_fecha = fecha.split('/')
    if len(partes_fecha) == 3:
        dia_normalizado = int(partes_fecha[2])
        mes_normalizado = int(partes_fecha[1])
        anio = partes_fecha[0]
        mes_anyo = f"{mes_normalizado}-{anio}" # Ej: "03-2017" se convierte en "3-2017"
        fecha = f"{anio}/{mes_normalizado}/{dia_normalizado}" # Ej: "2017-03-22" se convierte en "2017-3-22"
    else:
        print(f"Formato de fecha incorrecto: {fecha}")
        continue
```

- Recorremos las filas del archivo y dentro de este seleccionamos las columnas ‘Fecha’, ‘Medias’ y ‘Desviaciones’. Seguidamente dividimos el valor fecha en ‘mes_normalizado’ (normalizado para que sea 3 en vez de 03), ‘dia_normalizado’(igual que mes) y ‘anio’ y así crea la variable mes_anyo que necesitaremos más adelante y poder normalizar la variable fecha.

```
# 4. Insertar en DynamoDB
try:
    datosTable.put_item(
        Item={
            'mes_anyo': mes_anyo,
            'fecha': fecha,  #Partition key
            'media': media,
            'desviacion': desviacion
        }
    )
    items_processed += 1
except Exception as e:
    print(f"Error insertando {fecha}: {str(e)}")
```

- Intenta añadir los valores sacados del bucket en la tabla DynamoDB creada, sino es posible devolverá un error.

Para la segunda función le pondremos de nombre ‘proy-funcion-notificacion’ con las mismas características iniciales que la función anterior. A esta función le agregaremos como desencadenador la tabla DynamoDB creada para poder seleccionar los valores de desviación seleccionados, además le pondremos como tamaño del lote 500 para asegurarnos que coge todos los datos.

Esta función deberá mandar una notificación al usuario subscrito al tema creado cuando se encuentre una desviación superior a 0.5, de este modo le legará una notificación con la fecha y la desviación.

Arquitectura:

```
# For each inventory item added, check if the count is zero
for record in event['Records']:
    if record['eventName'] == 'INSERT':
        try:
            newImage = record['dynamodb']['NewImage']

            desviacion_str = newImage['desviacion']['S']
            fecha = newImage['fecha']['S']
            desviacion_val = float(desviacion_str)
```

- Revisa cada registro de eventos recibidos. Solo actuará si se añade un evento (record[‘eventName’] == ‘INSERT’). Intenta obtener la nueva imagen del dato insertado y saca la desviación (que convierte luego a número float) y la fecha, sino lo puede hacer devuelve un error

```
if desviacion_val > 0.5:
    mensaje = (
        f"ALERTA AQUASENSE:\n"
        f"Fecha: {fecha}\n"
        f"Desviación detectada: {desviacion_val}\n"
        f"Umbral permitido: 0.5"
    )
    sns = boto3.client('sns')

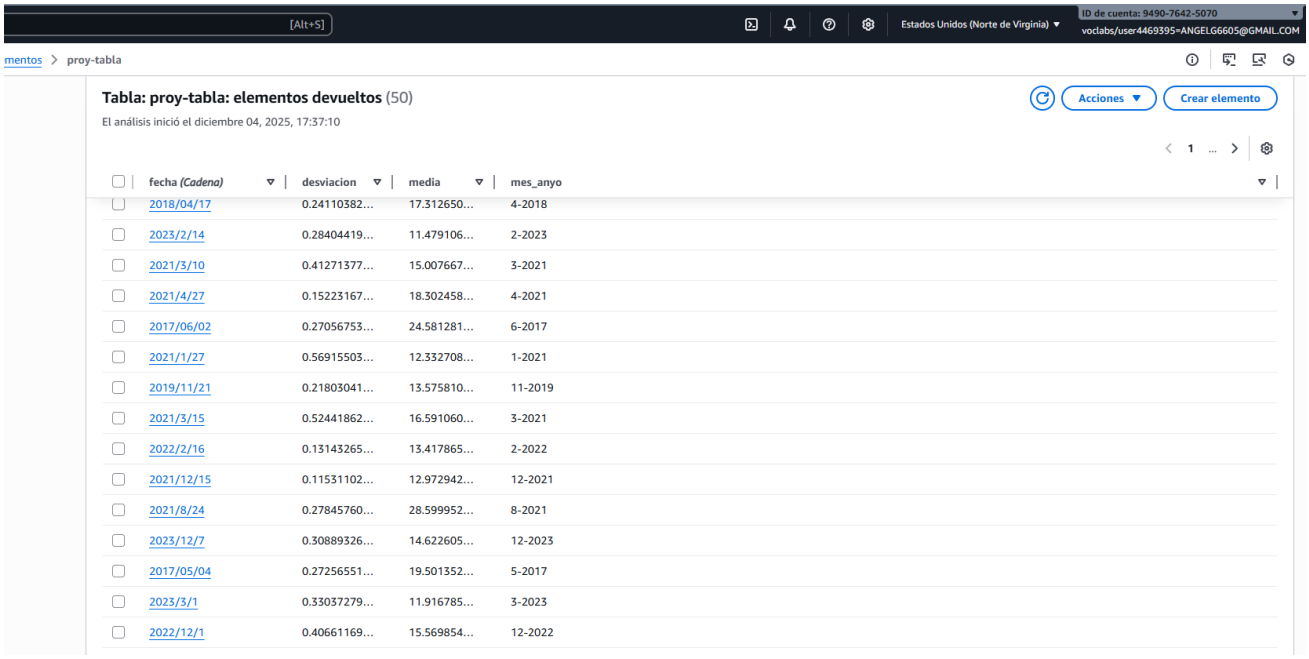
    alertTopic = 'proy-alarma-temperatura'

    snsTopicArn = [t['TopicArn'] for t in sns.list_topics()['Topics']
            if t['TopicArn'].lower().endswith(':' + alertTopic.lower())]
    for i in range(0, len(snsTopicArn)):

        sns.publish(
            TopicArn=snsTopicArn[i],
            Message=mensaje,
            Subject='¡Alerta Crítica Mar Menor!'
        )
        print(f"Alerta enviada para fecha {fecha}")
        alertas_enviadas += 1
```

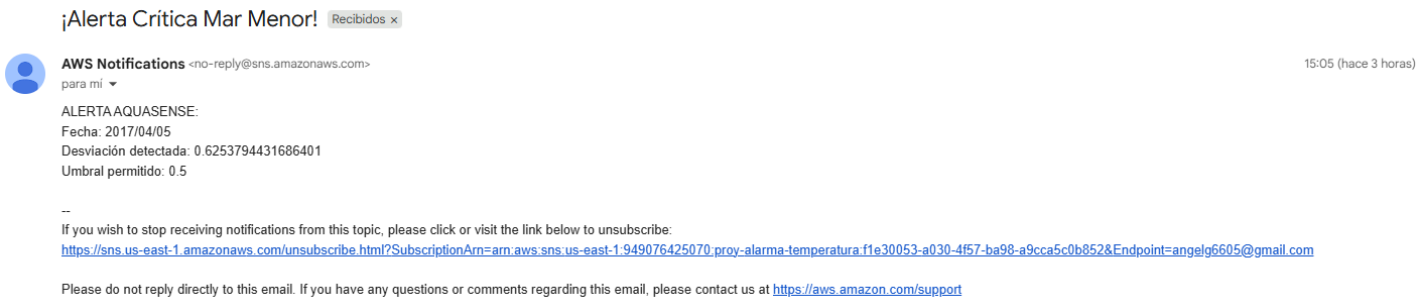
- Si la desviación es mayor que 0.5 (condición de ejecutar esta función) crea un mensaje con la fecha y la desviación detectada, y selecciona el tema creado en SNS y lo pública enviando este mensaje a todos los usuarios subscritos al tema creado.

Como hemos considerado diferenciar entre los aspectos del proyecto que conllevan a las practicas 2 y 4, ahora haremos una comprobación de que al llegar a este punto todo ha funcionado bien. Comenzamos añadiendo distintas particiones del archivo csv original para comprobar que la tabla no se sobrecarga.



fecha (Cadena)	desviacion	media	mes_anyo
2018/04/17	0.24110382...	17.312650...	4-2018
2023/2/14	0.28404419...	11.479106...	2-2023
2021/3/10	0.41271377...	15.007667...	3-2021
2021/4/27	0.15223167...	18.302458...	4-2021
2017/06/02	0.27056753...	24.581281...	6-2017
2021/1/27	0.56915503...	12.332708...	1-2021
2019/11/21	0.21803041...	13.575810...	11-2019
2021/3/15	0.52441862...	16.591060...	3-2021
2022/2/16	0.13143265...	13.417865...	2-2022
2021/12/15	0.11531102...	12.972942...	12-2021
2021/8/24	0.27845760...	28.599952...	8-2021
2023/12/7	0.30889326...	14.622605...	12-2023
2017/05/04	0.27256551...	19.501352...	5-2017
2023/3/1	0.33037279...	11.916785...	3-2023
2022/12/1	0.40661169...	15.569854...	12-2022

Como podemos ver en la captura se han cargado todos los datos (son 380 ya que hay algunas filas que están repetidas y se sobrescriben).



También en esta captura podemos ver que hemos recibido la notificación oportuna para un caso concreto en el que la desviación supera el umbral. En total llegan 36 correos por lo que suponemos que puede haber 36 casos en los que se supera este umbral.

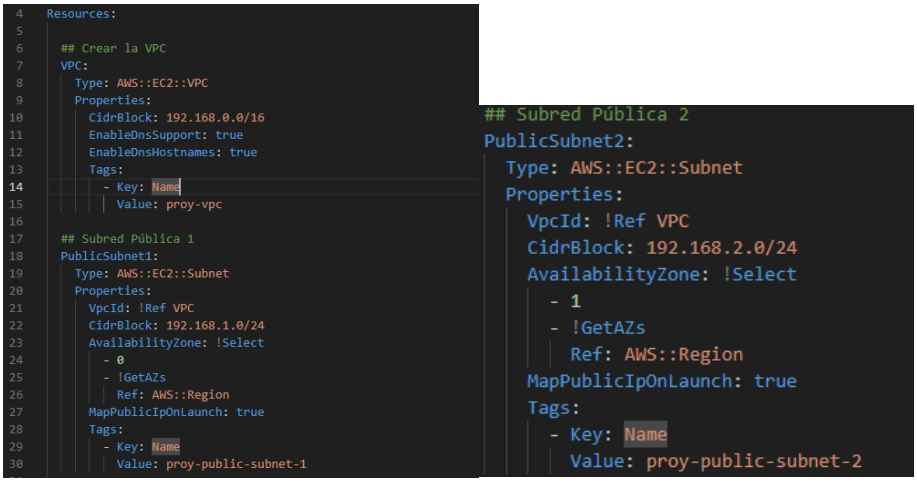
Tarea 2

Una vez realizado todo lo necesario para construir la arquitectura del pipeline, va a ser necesario crear un servicio web al que hacer las solicitudes, este servicio cogerá los datos provenientes de la tabla de Dynamodb creada anteriormente.

1º Creación de infraestructura básica a través de plantilla:

Accediendo a Cloudformation, creamos una nueva pila llamada proy-pila la cual creará lo establecido en la plantilla “proy-plantilla”.

- Empezamos creando una VPC de nombre proy-vpc la cual contendrá dos subredes publicas llamadas proy-public-subnet-1 y proy-public-subnet-2, estas subredes se encontrarán en zonas de disponibilidad distintas para garantizar alta disponibilidad y tolerancia a fallos.



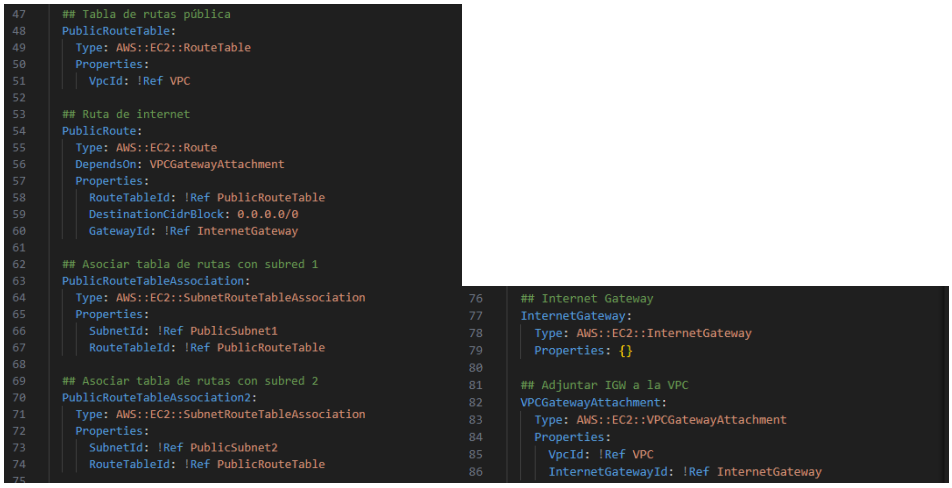
```
Resources:

  ## Crear la VPC
  VPC:
    Type: AWS::EC2::VPC
    Properties:
      CidrBlock: 192.168.0.0/16
      EnableDnsSupport: true
      EnableDnsHostnames: true
      Tags:
        - Key: Name
          Value: proy-vpc

  ## Subred Pública 1
  PublicSubnet1:
    Type: AWS::EC2::Subnet
    Properties:
      VpcId: !Ref VPC
      CidrBlock: 192.168.1.0/24
      AvailabilityZone: !Select
        - 0
        - !GetAZs
          Ref: AWS::Region
      MapPublicIpOnLaunch: true
      Tags:
        - Key: Name
          Value: proy-public-subnet-1
```

```
## Subred Pública 2
PublicSubnet2:
  Type: AWS::EC2::Subnet
  Properties:
    VpcId: !Ref VPC
    CidrBlock: 192.168.2.0/24
    AvailabilityZone: !Select
      - 1
      - !GetAZs
        Ref: AWS::Region
    MapPublicIpOnLaunch: true
    Tags:
      - Key: Name
        Value: proy-public-subnet-2
```

- Creamos una tabla de enrutamiento, la cual contendrá las reglas necesarias para el correcto tráfico de información, creando a su vez una ruta de acceso a internet Gateway. Asociamos las dos subredes a la tabla de enrutamiento, haciendo que sean accesibles por internet.



```
  ## Tabla de rutas pública
  PublicRouteTable:
    Type: AWS::EC2::RouteTable
    Properties:
      VpcId: !Ref VPC

  ## Ruta de internet
  PublicRoute:
    Type: AWS::EC2::Route
    DependsOn: VPCGatewayAttachment
    Properties:
      RouteTableId: !Ref PublicRouteTable
      DestinationCidrBlock: 0.0.0.0/0
      GatewayId: !Ref InternetGateway

  ## Asociar tabla de rutas con subred 1
  PublicRouteTableAssociation:
    Type: AWS::EC2::SubnetRouteTableAssociation
    Properties:
      SubnetId: !Ref PublicSubnet1
      RouteTableId: !Ref PublicRouteTable

  ## Asociar tabla de rutas con subred 2
  PublicRouteTableAssociation2:
    Type: AWS::EC2::SubnetRouteTableAssociation
    Properties:
      SubnetId: !Ref PublicSubnet2
      RouteTableId: !Ref PublicRouteTable
```

```
  ## Internet Gateway
  InternetGateway:
    Type: AWS::EC2::InternetGateway
    Properties: {}

  ## Adjuntar IGW a la VPC
  VPCGatewayAttachment:
    Type: AWS::EC2::VPCGatewayAttachment
    Properties:
      VpcId: !Ref VPC
      InternetGatewayId: !Ref InternetGateway
```

- Creamos tres grupos de seguridad cada uno con una función diferente:

FlaskSecurityGroup: Permite trafico entrante al puerto 5000 y al puerto 22, ira asociado a la instancia docker, permitiendo que esta sea accesible a través de internet (puerto 5000) y accesible a través de Putty gracias al puerto 22. Nos será necesario para el desarrollo de la aplicación Flask.

```
87  ##Grupo de seguridad asociado a la instancia
88  FlaskSecurityGroup:
89    Type: AWS::EC2::SecurityGroup
90    Properties:
91      GroupDescription: Permite conexion a la instancia
92      VpcId: !Ref VPC
93      SecurityGroupIngress:
94        - IpProtocol: tcp
95          FromPort: 5000
96          ToPort: 5000
97          CidrIp: 0.0.0.0/0
98        - IpProtocol: tcp
99          FromPort: 22
100         ToPort: 22
101         CidrIp: 0.0.0.0/0
102     Tags:
103       - Key: Name
104         Value: Flask Security Group
105
```

BalanceadorSecurityGroup: El principal objetivo es permitir que el trafico web estándar llegue al balanceador

TaskSecurityGroup: Este grupo de seguridad únicamente permite el tráfico en el puerto 5000 proveniente del balanceador de carga.

```
107 ## Security Group del Balanceador
108 BalanceadorSecurityGroup:
109   Type: AWS::EC2::SecurityGroup
110   Properties:
111     GroupName: BalanceadorSecurityGroup
112     GroupDescription: "Acceso HTTP publico para balanceador"
113     VpcId: !Ref VPC
114     SecurityGroupIngress:
115       - IpProtocol: tcp
116         FromPort: 80
117         ToPort: 80
118         CidrIp: 0.0.0.0/0
119     Tags:
120       - Key: Name
121         Value: BalanceadorSecurityGroup
```

```
123 ## Security Group de las tareas ECS (puerto 5000 desde el balanceador)
124 TaskSecurityGroup:
125   Type: AWS::EC2::SecurityGroup
126   DependsOn: BalanceadorSecurityGroup
127   Properties:
128     GroupName: TaskSecurityGroup
129     GroupDescription: "Permite trafico al puerto Flask 5000 desde el balanceador"
130     VpcId: !Ref VPC
131     SecurityGroupIngress:
132       - IpProtocol: tcp
133         FromPort: 5000
134         ToPort: 5000
135         SourceSecurityGroupId: !Ref BalanceadorSecurityGroup
136     Tags:
137       - Key: Name
138         Value: TaskSecurityGroup
139
```

- Finalmente, creamos una instancia de EC2 llamada DockerInstance con un par de claves asociado llamado proy-keypair, que habrá sido creado previamente. Como he mencionado antes, tendrá asociado el grupo de seguridad FlaskSecurityGroup.

La finalidad de hacer de esta instancia un docker, es hacerla un entorno de prueba y host de contenedores, aprovechando la portabilidad, el aislamiento y la eficiencia que ofrecen estos. DiskVolume básicamente servirá para expandir la capacidad del docker y DiskMountPoint para que el docker acceda a ese espacio.

```
## Instancia EC2 de pruebas
DockerInstance:
  Type: AWS::EC2::Instance
  Properties:
    InstanceType: t2.micro
    ImageId: ami-08a0d1e16fc3f61ea
    KeyName: proy-keypair
    IamInstanceProfile: LabInstanceProfile
    SubnetId: !Ref PublicSubnet1
    SecurityGroupIds:
      - !Ref FlaskSecurityGroup
    Tags:
      - Key: Name
        Value: DockerInstance
    UserData:
      Fn::Base64: !Sub |
        #!/bin/bash
        dnf update -y
        dnf install -y docker
        systemctl enable docker
        systemctl start docker
DiskVolume:
  Type: AWS::EC2::Volume
  Properties:
    Size: 100
    AvailabilityZone: !GetAtt DockerInstance.AvailabilityZone
  DeletionPolicy: Snapshot

DiskMountPoint:
  Type: AWS::EC2::VolumeAttachment
  Properties:
    InstanceId: !Ref DockerInstance
    VolumeId: !Ref DiskVolume
    Device: /dev/sdh
```

2º Creación de aplicación web:

El siguiente paso es comenzar a crear la aplicación Flask:

- Accedemos a putty con la Ip publica de DockerInstance y el par de claves proy-keypair.
- Creamos un nuevo directorio el cual contendrá la aplicación web flask llamado flask_docker. Este directorio tendrá tres ficheros: Dockerfile, requirements.txt y función.py

Funcion.py: El objetivo de la función Python contenido en este fichero será recibir solicitudes web y responder a éstas.

En este programa Python, primero importamos la clase Flask desde el paquete flask y lo utilizamos para crear una instancia de aplicación Flask llamada app.

Creo una interfaz orientada a objetos para interactuar con el servicio de DynamoDB. Este objeto (dynamodb) es el punto de partida para acceder a todas las tablas que existan en esa región. Una vez establecida la conexión accedo a la tabla que cree al principio 'proy-tabla'.

```
 GNU nano 5.8
from flask import Flask, request, jsonify
import boto3
from decimal import Decimal
from boto3.dynamodb.conditions import Attr

app = Flask(__name__)

tabla_dy = 'proy-tabla'
region = 'us-east-1'

dynamodb = boto3.resource('dynamodb', region_name=region)
table = dynamodb.Table(tabla_dy)
```

Creo una función auxiliar llamada “media” la cual nos será útil en la función “get_temp()”, toma como parámetros un diccionario llamado ítems y el campo al que queremos acceder del objeto en cuestión.

```
def media(items, field: str):
    """
    Calcula la media de una lista numérica almacenada en el atributo `field`
    (lista de Decimal / float / int).
    """
    # Convertimos cada valor a float por si vienen como Decimal
    valores = []
    for i in items:
        valores.append(float(i[field]))

    media_global = sum(valores) / len(valores)
    return media_global
```

El decorador @app.route transforma la función Python en una función “view” de Flask que convierte la respuesta de la función en una respuesta HTTP. Definiremos las rutas de la función /temp, /sd, /maxdiff las cuales permitirán acceso a las funciones en cuestión. Solo responderá a solicitudes GET, que son típicamente utilizadas para recuperar datos.

Esta linea recupera dos parámetros de la URL: year (año) y month (mes).

```
    y, m = request.args.get('year'), request.args.get('month')
```

Creamos una clave de búsqueda en el formato mes-año, que nos permitirá acceder a la columna “mes_anyo” de la tabla de dynamodb más fácilmente. Con “table.scan” se filtra para buscar solo aquellos ítems donde el atributo “mes-anyo” sea igual al valor buscado. Con “response.get('Items',[])” guardamos en un diccionario todos los ítems que coincidan. Para majear los fallos debidos a la solicitud de datos inexistentes en nuestra tabla agregamos una excepción.

```
    mes_anyo = f"{m}-{y}"
    response = table.scan(
        FilterExpression=Attr("mes_anyo").eq(mes_anyo)
    )
    items = response.get('Items',[])
    if not items:
        return jsonify({"error": f"Datos no encontrados para {mes_anyo}"}, 404)
```

- get_temp()

Esta función le aplica la función media al campo “media” a todos los datos de un mes, devolviéndonos la media de temperatura de un mes.

```python
@app.route('/temp', methods=['GET'])
def get_temp():
    y, m = request.args.get('year'), request.args.get('month')
    mes_anyo = f"{m}-{y}"
    response = table.scan(
        FilterExpression=Attr("mes_anyo").eq(mes_anyo)
    )
    items = response.get('Items',[])
    if not items:
        return jsonify({"error": f"Datos no encontrados para {mes_anyo}"}, 404)

    media_temp = media(items, "media")

    return jsonify({
    f"Media de temperatura del mes {mes_anyo}": media_temp
    })
```

- get_maxdiff():

En esta función va a calcular la máxima diferencia de temperatura que existe entre el mes solicitado y el mes inmediatamente anterior. La función primero calcula el máximo del primer mes y luego el máximo del segundo, luego calcula la diferencia.

```
 GNU nano 5.8                                                    funcion.py
@app.route('/maxdiff', methods=['GET'])
def get_maxdiff():
    y, m = int(request.args.get('year')), int(request.args.get('month'))
    mes_anyo = f"{m}-{y}"
    response = table.scan(
        FilterExpression=Attr("mes_anyo").eq(mes_anyo)
    )
    items = response.get('Items',[])
    if not items:
        return jsonify({"error": f"Datos no encontrados para {mes_anyo}"}, 404)

    max_act = 0
    for i in items:
        if float(i['media']) > max_act:
            max_act = float(i['media'])

    if m == 1:
        m_2 = 12
        y_2 = y-1

    else:
        m_2 = m-1
        y_2 = y

    mes_anyo_ant = f"{m_2}-{y_2}"
    response_ant = table.scan(
        FilterExpression=Attr("mes_anyo").eq(mes_anyo_ant)
    )
    items_ant = response_ant.get('Items',[])
    if not items_ant:
        return jsonify({"error": f"Datos no encontrados para mes anterior"}, 404)

    max_ant = 0
    for i in items_ant:
        if float(i['media']) > max_ant:
            max_ant = float(i['media'])

    max_diff = max_act-max_ant
    return jsonify({ f"Diferencia maxima de temperatura del mes {mes_anyo}": max_diff
```

- get_sd():

Esta función devuelve el valor máximo del campo “desviación” de un mes:

```
@app.route('/sd', methods=['GET'])
def get_sd():
    y, m = request.args.get('year'), request.args.get('month')
    mes_anyo = f"{m}-{y}"
    response = table.scan(
        FilterExpression=Attr("mes_anyo").eq(mes_anyo)
    )

    items = response.get('Items',[])
    if not items:
        return jsonify({"error": f"Datos no encontrados para {mes_anyo}"}, 404)

    max = 0
    for i in items:
        if float(i['desviacion']) > max:
            max = float(i['desviacion'])

    return jsonify({
    f"Maxima desviacion del mes {mes_anyo}": max
})
```

- health_check():

Este es un endpoint de salud, utilizado para comprobar si la aplicación esta en funcionamiento.

```
@app.route('/health', methods=['GET'])
def health_check():
    return jsonify({'status': 'healthy'}), 200
```

Las dos últimas líneas de este programa garantizan por un lado que si el puerto 5000 está ya ocupado por otra aplicación el programa siga ejecutándose en otro puerto. Por otro lado, la aplicación Flask se ejecuta en modo depuración y es accesible desde el exterior

```
if __name__ == '__main__':
    app.run(host='0.0.0.0', port=5000)
```

*Es posible que al copiar la función este código de error, por lo que podría ser necesario reescribirlo a mano.

requirements.txt: Fichero que contiene todas las dependencias de la aplicación Flask:

```
  GNU nano 5.8
Flask==2.1.1
boto3==1.24.5
werkzeug==2.1.2
Jinja2==3.1.2
MarkupSafe==2.1.1
itsdangerous==2.1.2
click==8.1.3
```

Dockerfile:

- Utiliza una imagen base basada en Linux Alpine (debido a su tamaño reducido) que será el sistema operativo en el que instalaremos nuestra imagen.
- Establece el directorio /app como el directorio de trabajo de nuestra aplicación en el contenedor.
- Copia el fichero requirements.txt en el directorio de trabajo del contenedor.
- Instala todos los paquetes indicados en el fichero requirements.txt.
- Copia todo el contenido del directorio actual y el directorio de trabajo.
- Indica el comando que ejecutará el contenedor.

```
  GNU nano 5.8                                                    Dockerfile
# Usamos la imagen base de Python 3.8 sobre Alpine
FROM python:3.8-alpine

# Establecemos el directorio de trabajo dentro del contenedor
WORKDIR /app

# Copiamos el archivo requirements.txt al contenedor
COPY requirements.txt requirements.txt

# Instalamos las dependencias listadas en el requirements.txt
RUN pip install --no-cache-dir -r requirements.txt

# Copiamos todo el contenido de la carpeta local al contenedor
COPY . .

# Especificamos el comando para ejecutar el script Python metadata.py al inicia>
CMD ["python3", "funcion.py"]
```

3º Prueba de funcionamiento:

- Construimos la imagen y creamos el contendor en el puerto 5000

```
[ec2-user@ip-192-168-1-33 flask_docker]$ sudo docker build -t flask_container .
[+] Building 0.2s (10/10) FINISHED
 => [internal] load build definition from Dockerfile
 => => transferring dockerfile: 641B
 => [internal] load metadata for docker.io/library/python:3.8-alpine
 => [internal] load .dockerignore
 => => transferring context: 2B
 => [1/5] FROM docker.io/library/python:3.8-alpine@sha256:3d93b1f77efce339aa77db726656872517b0d67837989aa7c4b35bd5ae7e81ba
 => [internal] load build context
 => => transferring context: 368B
 => CACHED [2/5] WORKDIR /app
 => CACHED [3/5] COPY requirements.txt requirements.txt
 => CACHED [4/5] RUN pip install --no-cache-dir -r requirements.txt
 => CACHED [5/5] COPY . .
 => exporting to image
 => => exporting layers
 => => writing image sha256:64d6e714ac76fd323c2a3d20e8ab73e7485f2a456d0b68596dff5a8f640630db
 => => naming to docker.io/library/flask_container
[ec2-user@ip-192-168-1-33 flask_docker]$ sudo docker run -p 5000:5000 -d flask_container
d2192de50a78b358efe7655af54771dc7b77af100c1df799960620eb061aa614
```

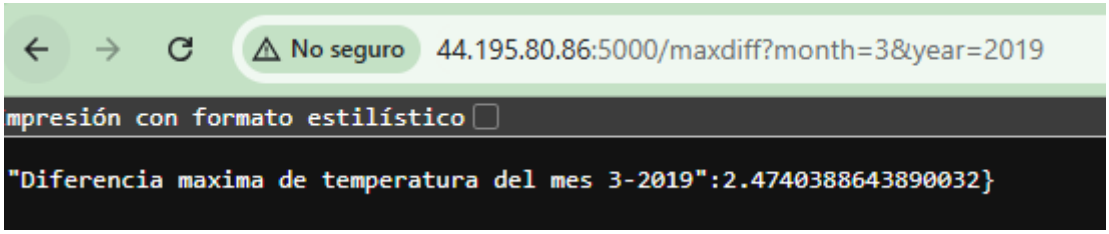
- Comprobamos que se han creado correctamente:

```
[ec2-user@ip-192-168-1-33 flask_docker]$ sudo docker ps
CONTAINER ID   IMAGE             COMMAND              CREATED          STATUS          PORTS                                       NAMES
d2192de50a78   flask_container   "python3 funcion.py"   14 seconds ago   Up 13 seconds   0.0.0.0:5000->5000/tcp, :::5000->5000/tcp   heuristic_maxwell
```

```
[ec2-user@ip-192-168-1-33 flask_docker]$ sudo docker images
REPOSITORY         TAG       IMAGE ID       CREATED              SIZE
flask_container    latest    64d6e714ac76   23 minutes ago       132MB
```

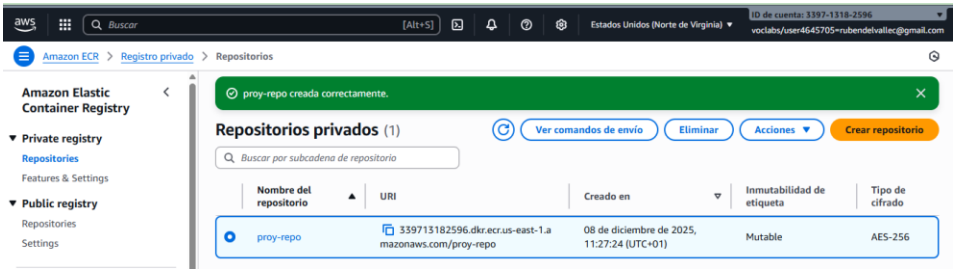
- Comprobamos que todas las funciones funcionan correctamente tanto en la terminal como desde internet:

```
[ec2-user@ip-192-168-1-33 flask_docker]$ curl "http://127.0.0.1:5000/sd?month=10&year=2019"
{"Maxima desviacion del mes 10-2019":0.6360582330690658}
[ec2-user@ip-192-168-1-33 flask_docker]$ curl "http://127.0.0.1:5000/temp?month=10&year=2019"
{"Media de temperatura del mes 10-2019":22.658597559121585}
[ec2-user@ip-192-168-1-33 flask_docker]$ curl "http://127.0.0.1:5000/maxdiff?month=10&year=2019"
{"Diferencia maxima de temperatura en mes 10-2019":5.310085582610821}
[ec2-user@ip-192-168-1-33 flask_docker]$
```



4º Creación del repositorio al que subir la imagen

- Como paso previo a la tarea3, creamos un repositorio ECR al cual he llamado “proy-repo”, cuya funcionalidad será almacenar, gestionar y distribuir imágenes de contenedores.



Subimos la imagen al repositorio:

- Recupero un token de autenticación que autentique nuestro cliente de Docker en el registro.

```
[ec2-user@ip-192-168-1-33 ~]$ aws ecr get-login-password --region us-east-1 | do
cker login --username AWS --password-stdin 339713182596.dkr.ecr.us-east-1.amazon
aws.com
```

- Etiqueto la imagen para poder enviarla a este repositorio:

```
[ec2-user@ip-192-168-1-33 ~]$ docker tag flask_container 339713182596.dkr.ecr.us
-east-1.amazonaws.com/proy-repo:flask_container
```

- Envió esta imagen al repositorio de AWS recién creado

```
[ec2-user@ip-192-168-1-33 ~]$ docker push 339713182596.dkr.ecr.us-east-1.amazona
ws.com/proy-repo:flask_container
```

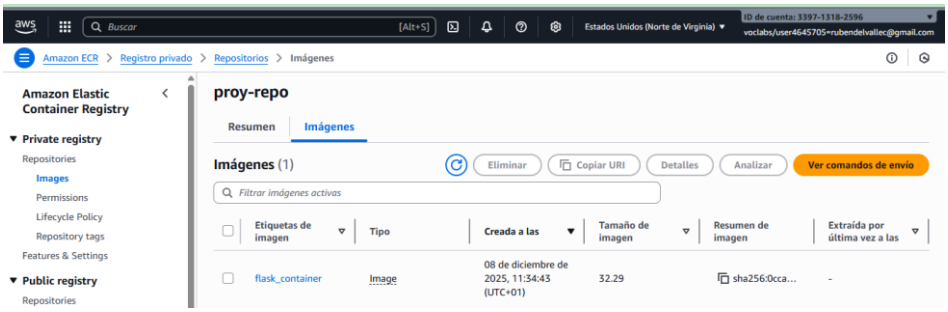
```
[ec2-user@ip-192-168-1-33 ~]$ aws ecr get-login-password --region us-east-1 | do
cker login --username AWS --password-stdin 339713182596.dkr.ecr.us-east-1.amazon
aws.com
WARNING! Your password will be stored unencrypted in /home/ec2-user/.docker/conf
ig.json.
Configure a credential helper to remove this warning. See
https://docs.docker.com/engine/reference/commandline/login/#credentials-store

Login Succeeded
[ec2-user@ip-192-168-1-33 ~]$ docker tag flask_container 339713182596.dkr.ecr.us
-east-1.amazonaws.com/proy-repo:flask_container
[ec2-user@ip-192-168-1-33 ~]$ docker push 339713182596.dkr.ecr.us-east-1.amazona
ws.com/proy-repo:flask_container
The push refers to repository [339713182596.dkr.ecr.us-east-1.amazonaws.com/proy
-repo]
746cc873e8d6: Pushed
9ad62875e95c: Pushed
6c677fe46a63: Pushed
268865912762: Pushed
817fa7f5a4af: Pushed
6b49371d7ed7: Pushed
50acbb6dc641: Pushed
63ca1fbb43ae: Pushed
flask_container: digest: sha256:0cca7ca33b52a8f336a0877a62ca51fbbbd912dd272a4156
0c4f48a2f6b904f2 size: 1990
```

*Es posible que al intentar acceder a nuestro token de autentificación debido a que el sistema entiende que no tenemos premisos, por lo que puede ser util copiar y volver a ingresar a esta ahora si con permisos:

sudo usermod -aG docker ec2-user

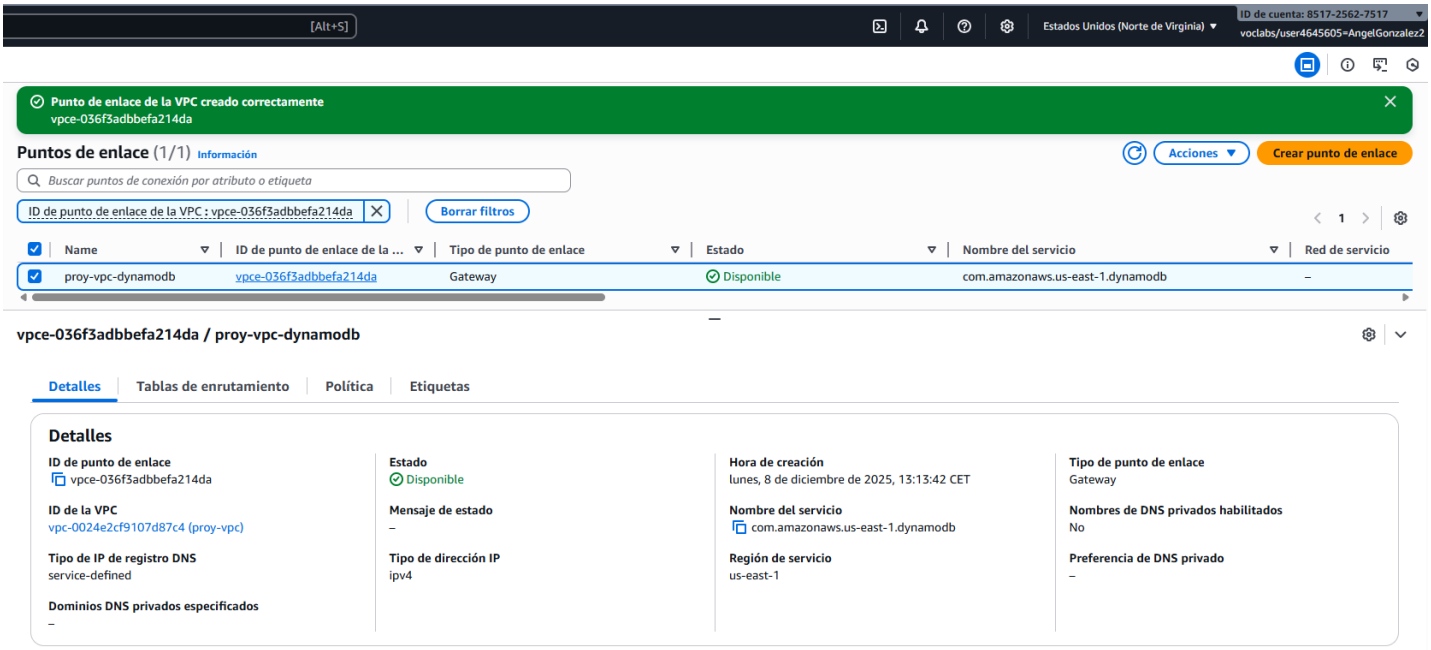
Finalmente, comprobamos que se ha subido correctamente.



Tarea 3

Ahora deberemos ocuparnos de crear e implementar las aplicaciones en un entorno de contenedores. Para esto usaremos la imagen de Docker creada.

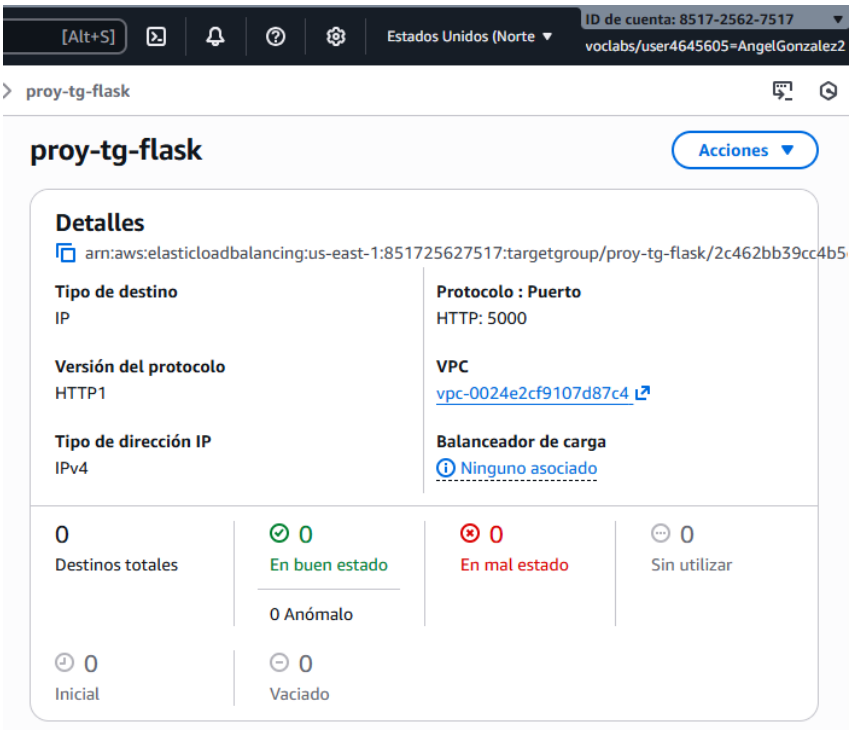
Primero deberemos crear **una puerta de enlace(Punto de conexión)** entre la vpc y dynamo DB para facilitar el paso de información de la tabla 'proy-tabla'. Debemos entrar en puntos de conexión en VPC y crear uno. Le llamaremos 'proy-vpc-dynamodb'. La categoría del servicio será Servicio AWS. En Servicios deberemos buscar dynamo y seleccionaremos la opción 'com.amazonaws.us-east-1.dynamodb' con el tipo Gateway. Para finalizar seleccionaremos la vpc 'proy-vpc' y todas las tablas de enrutamiento que aparecen (dos).



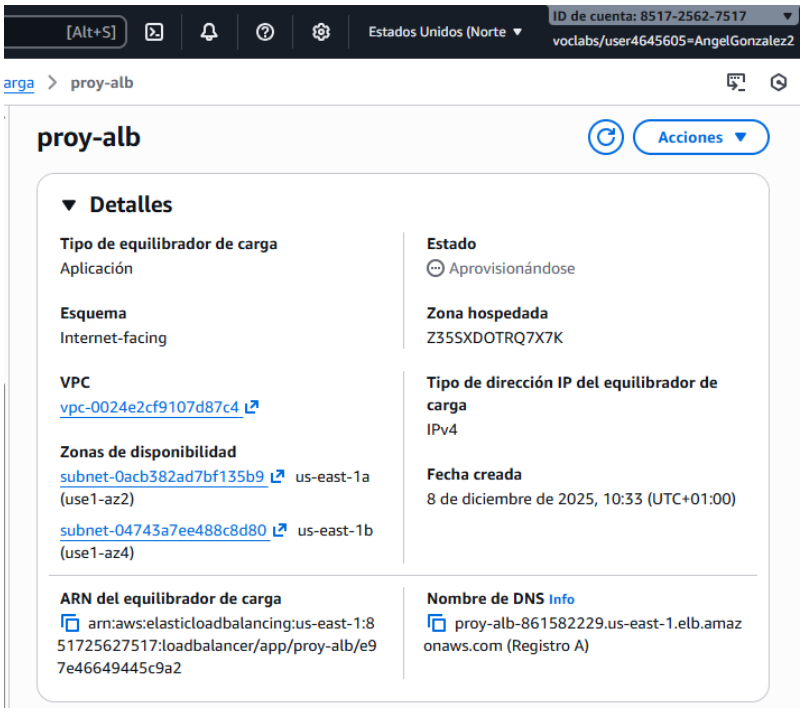
Comenzamos creando el **balanceador de carga** al cual llamaremos 'proy-alb'. Será un balanceador de carga de aplicaciones ya que vamos a usar un tráfico HTTP y HTTPS. Estará expuesto a Internet. Necesitamos que el balanceador escale dentro de una vpc, así que elegiremos la creada por la plantilla. Las zonas de disponibilidad deberán ser las dos subredes conectadas a la vpc. Claramente añadiremos como grupo de seguridad el que está relacionado con el balanceador (BalanceadorSecurityGroup). Crearemos el protocolo http y puerto 80 para el agente de escucha.

Finalmente faltaría añadir un **grupo de destino**, pero como no tenemos ninguno creado deberemos hacerlo. El grupo de destino se llamará 'proy-tg-flask'. El tipo de destino serán direcciones IP, esto lo hacemos ya usamos el modo de red awsvpc y cada contenedor tendrá su propia IP. El protocolo será HTTP y con puerto 5000 para coincidir con el puerto donde se escucha la aplicación Flask. La vpc (aunque está automáticamente seleccionada) será la creada en la plantilla. En la pestaña de comprobación de estado deberemos añadir 'health' a la ruta, ya que como hemos visto en la función de la tarea anterior nuestra función health_check que hace una comprobación de estado funciona con esta ruta. Finalmente deberemos modificar el umbral en buen estado como 2 (con 2

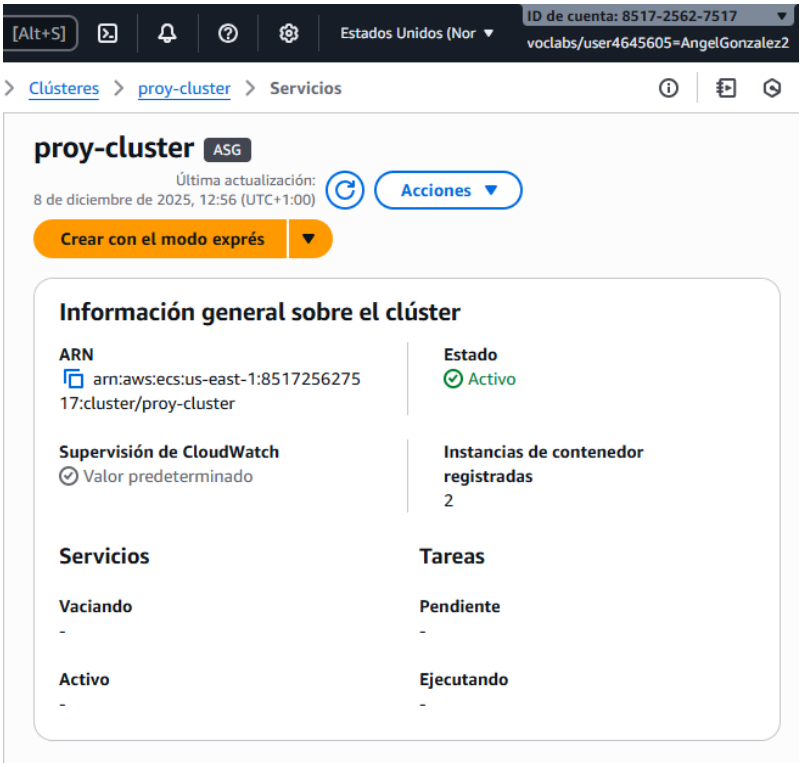
respuestas buenas seguidas el balanceador empieza a funcionar) y el intervalo a 6 (de este modo se realizan comprobaciones rápidas para detectar fallos o recuperaciones rápido)



Finalmente seleccionamos el grupo de destino creado para el balanceador y terminamos la implementación de este.

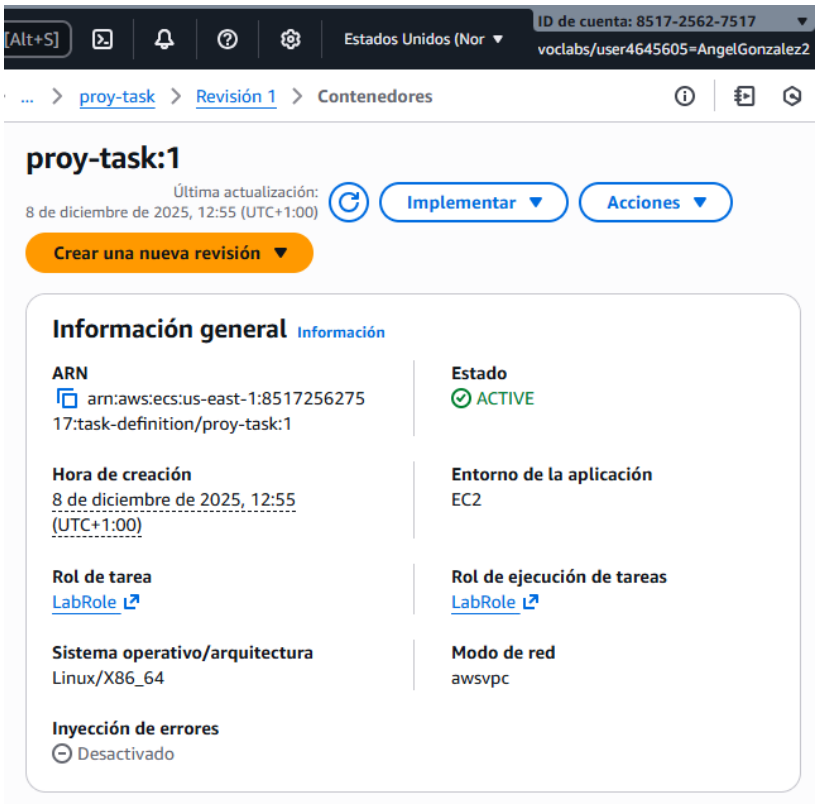


A continuación crearemos el **clúster** con nombre ‘proy-cluster’. La infraestructura será *Fargate e instacias autoadministrativas* . El modelo de aprovisionamiento deberá ser Bajo Demanda. El sistema operativo se queda como Amazon Linux 2023, pero el tipo de instancia será t2.micro ya que este tamaño es suficiente y económico para los contenedores de este proyecto. El rol de la instancia deberá ser LabInstanceProfile. En las capacidades deseadas la mínima debe ser 2 (garantiza que si una instancia falla, el servicio continúa con la otra) y la máxima 4 (Permite crecer automáticamente ante picos de demanda). Seleccionamos el par de claves creado para la pila y las vpc y subredes creadas en ella. Finalmente seleccionamos el grupo de seguridad TaskSecurityGroup y en la opción de asignar automáticamente una IP pública seleccionamos Encender, si no habilitamos esta opción, la instancia no tendrá una dirección IP accesible desde internet, por lo que no podremos conectarnos a ella mediante SSH ni ver la página web desde el navegador.



Seguidamente creamos la **definición de tareas** con nombre ‘proy-task’, el tipo de lanzamiento será únicamente Amazon EC2 ya que usamos un clúster con instancias EC2. Debemos confirmar que el modo de red es awsvpc ya que otorga a la tarea las mismas propiedades de red que las instancias EC2. Los tamaños serán 1 vCPU y .2 GB ya que la instancia t2.micro tiene recursos limitados y debemos dejar un margen para el sistema operativo. El rol de tarea y de ejecución corresponderán al LabRole.

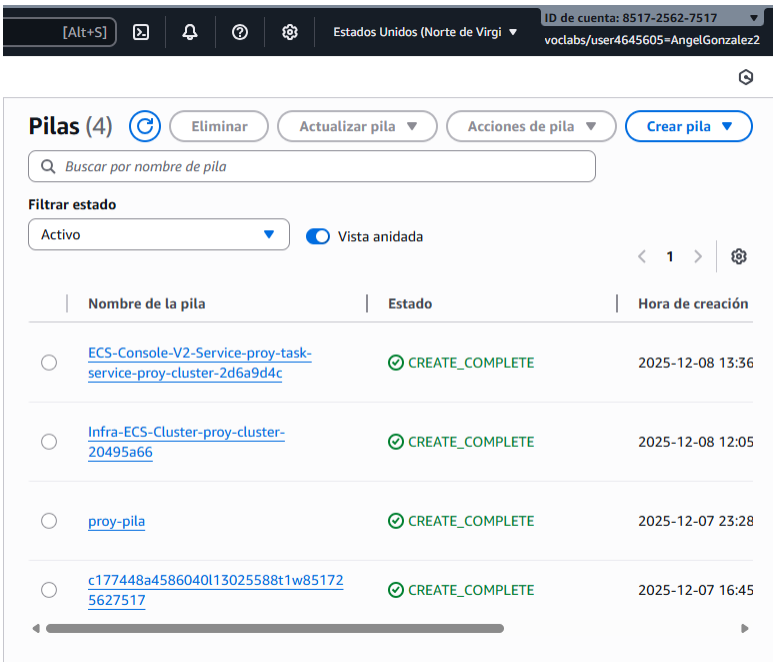
Deberemos detallar el **contenedor**, al que llamaremos ‘proy-container’. Copiamos en el apartado ‘Imagen’ la URI de la imagen que tenemos en el repositorio. En el mapeo de puertos el puerto del contenedor deberá ser 5000 y el protocolo TCP. Finalmente, los límites de asignación serán para la CPU 1, el límite estricto de memoria .2 GB y el límite flexible de memoria .1 GB (el GPU no lo tocamos).



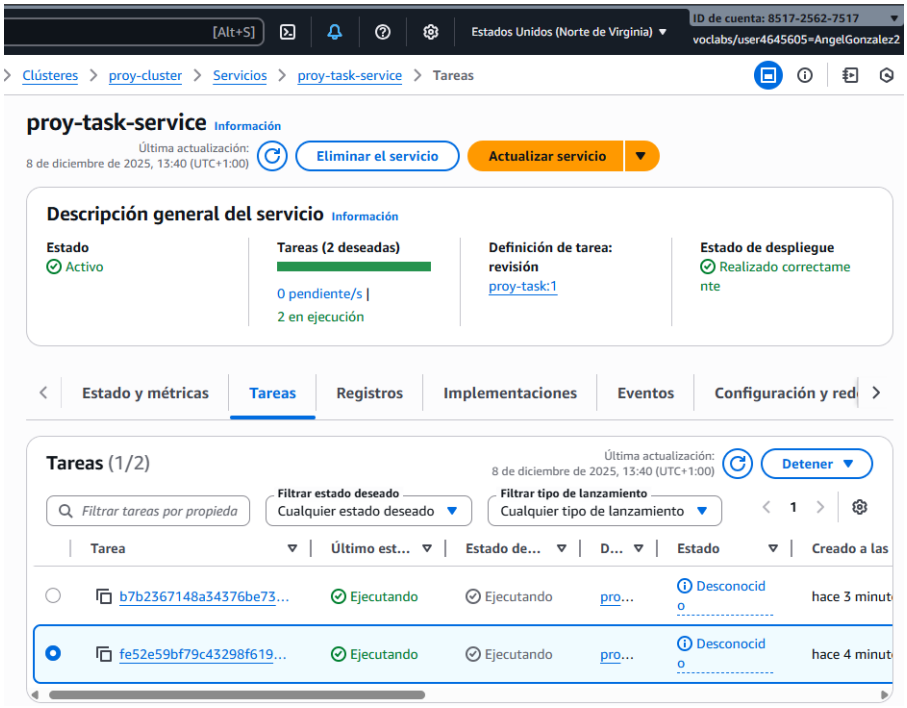
Finalmente creamos el **servicio**, este lo haremos dentro del clúster creado anteriormente. Le llamaremos ‘proy-ecs-service’. Deberemos seleccionar la definición de tareas ‘proy-task’ creada anteriormente. Como tipo de lanzamiento seleccionaremos EC2. El tipo de servicio será RÉPLICA ya que así mantendremos el número de tareas que hemos seleccionado en el clúster si hay algún fallo. Las tareas deseadas son 2 hecha para la tolerancia a fallos. Seleccionamos la vpc creada y las subredes relacionadas. El grupo se seguridad deberá ser TaskSecurityGroup ya que este grupo solo permite tráfico en el puerto 5000 proveniente del Balanceador de Carga. A continuación debemos seleccionar en la pestaña ‘Equilibrio de carga’ que vamos a usar uno. Será un balanceador de carga de aplicaciones y en contenedor seleccionamos el creado ‘proy-container 5000:5000’, el balanceador de carga será ‘proy-alb’, el agente de escucha el ya creado ‘HTTP:80’ y el grupo de destino será ‘proy-tg-flask’.

Ahora empezamos a hacer comprobaciones de que todo ha ido bien:

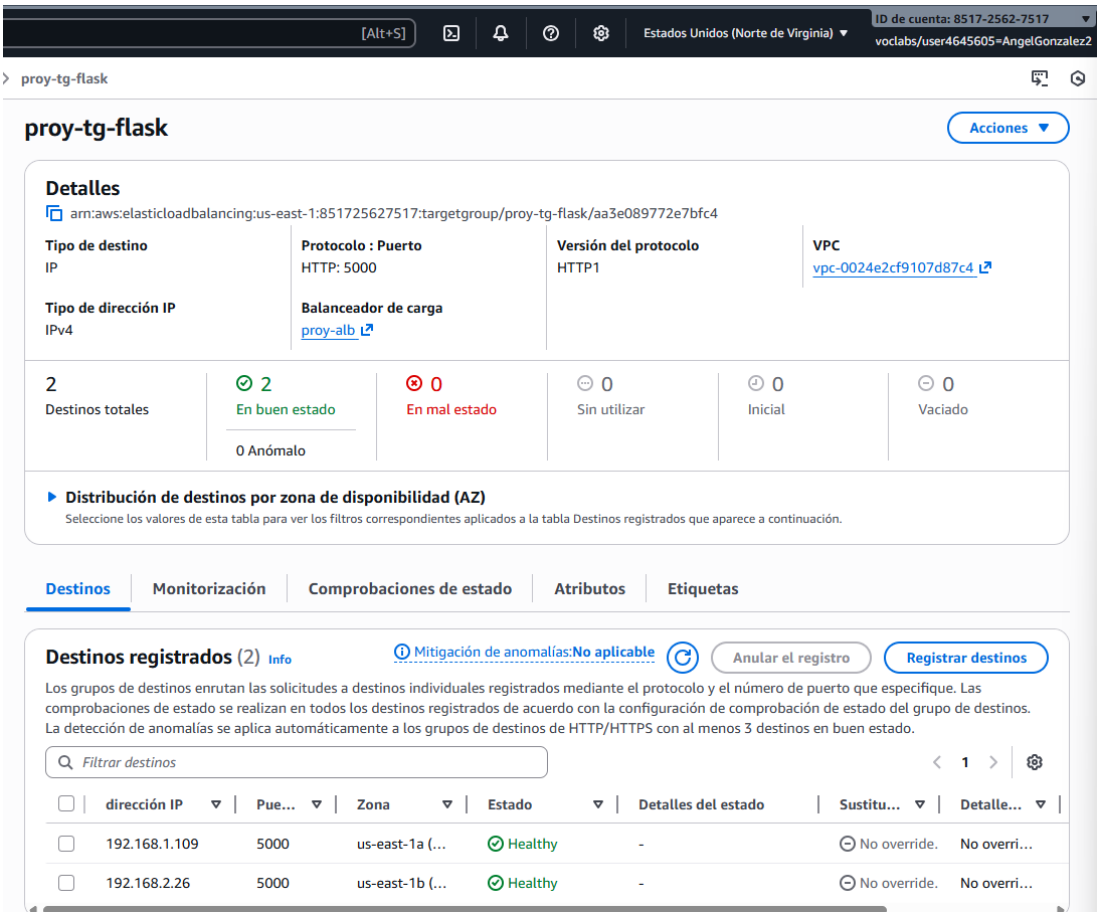
- Las pilas relacionadas con el servicio y el clúster se han creado sin problemas



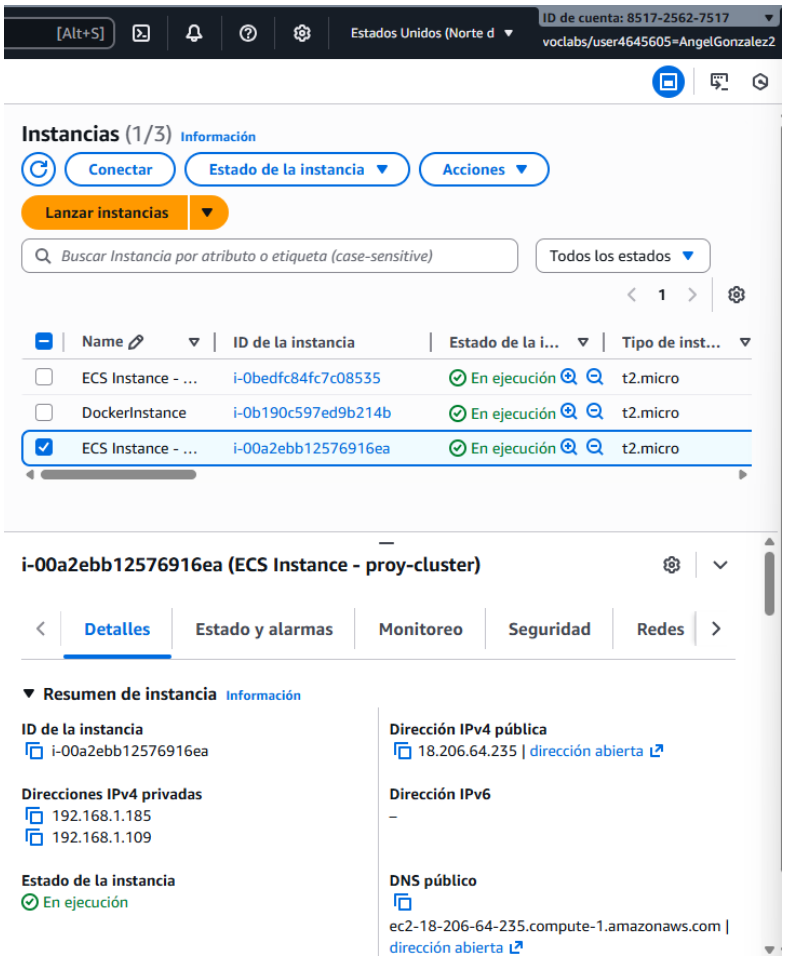
- Las tareas se han ejecutado correctamente y el servicio se mantiene activo



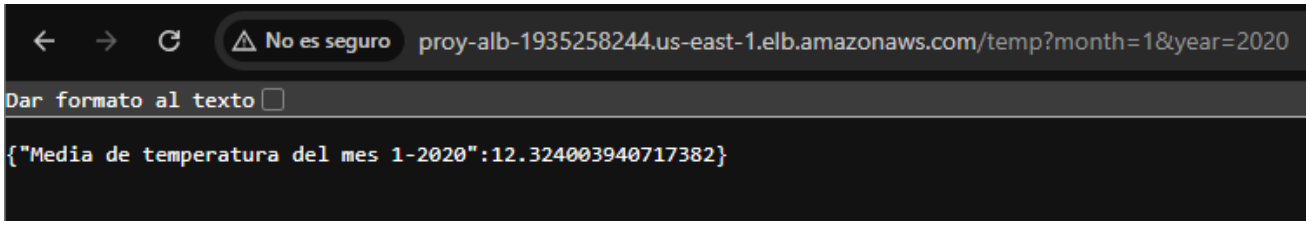
- El grupo de destino tiene dos destinos en buen estado

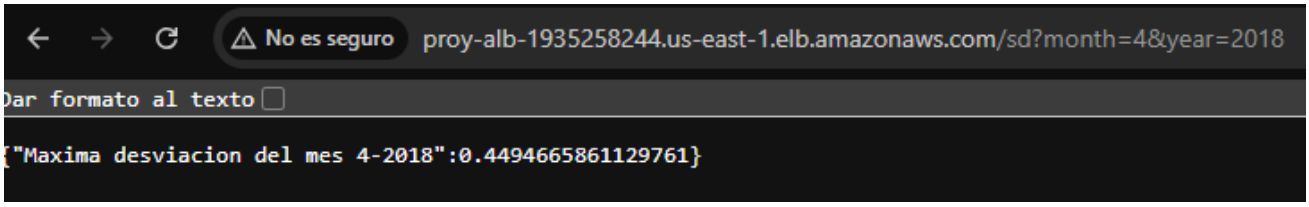


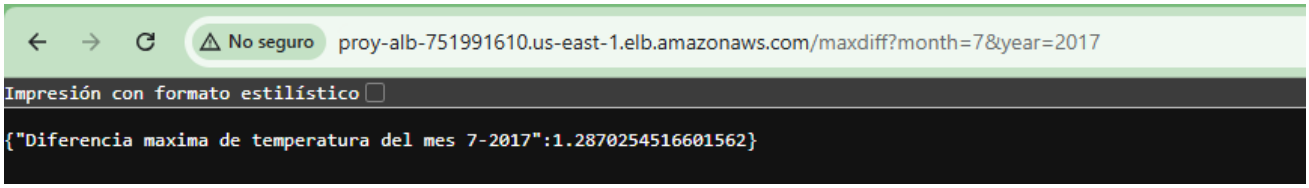
- Se han creado dos instancias, si además entramos a las dos instancias las direcciones IP de los destinos registrados son las direcciones IPv4 privadas



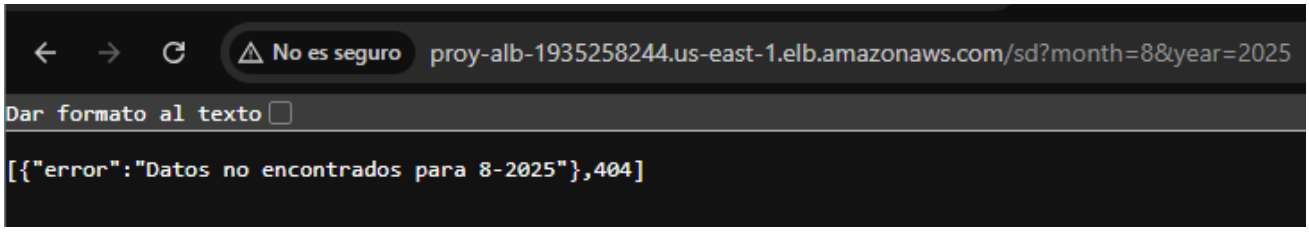
- Finalmente iremos solicitando en el navegador que todo va bien:

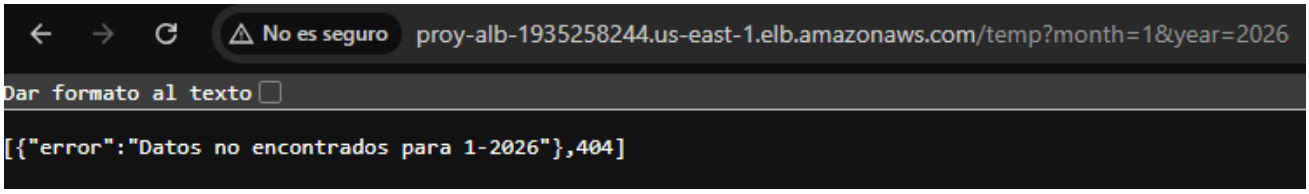






- También lo hacemos para los errores







Anexo de automatización

- **Función lambda 'Proy-funcion-procesador-datos':**

```
import json, urllib.parse
import boto3
import csv
import io
import os

# Clientes de AWS
s3 = boto3.client('s3')
dynamodb = boto3.resource('dynamodb')
datosTable = dynamodb.Table('proy-tabla')

def lambda_handler(event, context):

    # 1. Obtener detalles del archivo subido desde el evento S3
    bucket_name = event['Records'][0]['s3']['bucket']['name']
    file_key = urllib.parse.unquote_plus(event['Records'][0]['s3']['object']['key'])

    print(f"Procesando archivo: {file_key} del bucket: {bucket_name}")
```

```
    # 2. Leer el archivo CSV desde S3
    response = s3.get_object(Bucket=bucket_name, Key=file_key)
    content = response['Body'].read().decode('utf-8')

    # 3. Parsear el CSV
    csv_reader = csv.DictReader(io.StringIO(content), delimiter=',') # Ajusta
delimitador si es necesario

    items_processed = 0

    for row in csv_reader:
        # Formato esperado del CSV: Fecha, Medias, Desviaciones
        fecha = row['Fecha'] # Ej: 22/03/2017
        media = row['Medias']
        desviacion = row['Desviaciones']


        partes_fecha = fecha.split('/')
        if len(partes_fecha) == 3:
            dia_normalizado = int(partes_fecha[2])
            mes_normalizado = int(partes_fecha[1])
            anio = partes_fecha[0]
            mes_anyo = f"{mes_normalizado}-{anio}" # Ej: "03-2017" se convierte en
"3-2017"
            fecha = f"{anio}/{mes_normalizado}/{dia_normalizado}" # Ej: "2017/03/22"
se convierte en "2017/3/22"
        else:
            print(f"Formato de fecha incorrecto: {fecha}")
            continue

        # 4. Insertar en DynamoDB
        try:
            datosTable.put_item(
                Item={
                    'fecha': fecha,          # Partition Key
                    'mes_anyo': mes_anyo,
                    'media': media,
                    'desviacion': desviacion
                }
            )
            items_processed += 1
        except Exception as e:
            print(f"Error insertando {fecha}: {str(e)}")

    return {
```

```
        'statusCode': 200,
        'body': json.dumps(f'Procesados {items_processed} registros.')
```

- **Función lambda 'Proy-funcion-notificacion':**

```
# Stock Check Lambda function
#
# This function is triggered when values are inserted into the Inventory DynamoDB
table.
# Inventory counts are checked and if an item is out of stock, a notification is
sent to an SNS Topic.
import json, boto3

# This handler is run every time the Lambda function is triggered
def lambda_handler(event, context):
    # Show the incoming event in the debug log
    print("Event received by Lambda function: " + json.dumps(event, indent=2))
    alertas_enviadas = 0
    # For each inventory item added, check if the count is zero
    for record in event['Records']:
        if record['eventName'] == 'INSERT':
            try:
                newImage = record['dynamodb']['NewImage']

                desviacion_str = newImage['desviacion']['S']
                fecha = newImage['fecha']['S']
                desviacion_val = float(desviacion_str)

                if desviacion_val > 0.5:
                    mensaje = (
                        f"ALERTA AQUASENSE:\n"
                        f"Fecha: {fecha}\n"
                        f"Desviación detectada: {desviacion_val}\n"
                        f"Umbral permitido: 0.5"
                    )
                    sns = boto3.client('sns')
                    alertTopic = 'proy-alarma-temperatura'

                    snsTopicArn = [t['TopicArn'] for t in
sns.list_topics()['Topics']
```

```
                            if t['TopicArn'].lower().endswith(':' +
alertTopic.lower())]
                    for i in range(0, len(snsTopicArn)):

                        sns.publish(
                            TopicArn=snsTopicArn[i],
                            Message=mensaje,
                            Subject='¡Alerta Crítica Mar Menor!'
                        )
                        print(f"Alerta enviada para fecha {fecha}")
                        alertas_enviadas += 1


            except KeyError as e:
                print(f"Falta algún campo en el registro: {e}")
            except ValueError as e:
                print(f"Error convirtiendo desviación a número: {e}")

    # Finished!
    return 'Successfully processed {} records.'.format(len(event['Records']))
```

- **Pila CloudFormation :**

```
AWSTemplateFormatVersion: '2010-09-09'
Description: Crear una VPC con dos subredes públicas, grupos de seguridad y una
instancia EC2

Resources:

  ## Crear la VPC
  VPC:
    Type: AWS::EC2::VPC
    Properties:
      CidrBlock: 192.168.0.0/16
      EnableDnsSupport: true
      EnableDnsHostnames: true
      Tags:
        - Key: Name
          Value: proy-vpc

  ## Subred Pública 1
  PublicSubnet1:
    Type: AWS::EC2::Subnet
    Properties:
```

```
    VpcId: !Ref VPC
    CidrBlock: 192.168.1.0/24
    AvailabilityZone: !Select
      - 0
      - !GetAZs
        Ref: AWS::Region
    MapPublicIpOnLaunch: true
    Tags:
      - Key: Name
        Value: proy-public-subnet-1

## Subred Pública 2
PublicSubnet2:
  Type: AWS::EC2::Subnet
  Properties:
    VpcId: !Ref VPC
    CidrBlock: 192.168.2.0/24
    AvailabilityZone: !Select
      - 1
      - !GetAZs
        Ref: AWS::Region
    MapPublicIpOnLaunch: true
    Tags:
      - Key: Name
        Value: proy-public-subnet-2

## Tabla de rutas pública
PublicRouteTable:
  Type: AWS::EC2::RouteTable
  Properties:
    VpcId: !Ref VPC

## Ruta de internet
PublicRoute:
  Type: AWS::EC2::Route
  DependsOn: VPCGatewayAttachment
  Properties:
    RouteTableId: !Ref PublicRouteTable
    DestinationCidrBlock: 0.0.0.0/0
    GatewayId: !Ref InternetGateway

## Asociar tabla de rutas con subred 1
PublicRouteTableAssociation:
  Type: AWS::EC2::SubnetRouteTableAssociation
  Properties:
```

```
      SubnetId: !Ref PublicSubnet1
      RouteTableId: !Ref PublicRouteTable

  ## Asociar tabla de rutas con subred 2
  PublicRouteTableAssociation2:
    Type: AWS::EC2::SubnetRouteTableAssociation
    Properties:
      SubnetId: !Ref PublicSubnet2
      RouteTableId: !Ref PublicRouteTable

  ## Internet Gateway
  InternetGateway:
    Type: AWS::EC2::InternetGateway
    Properties: {}

  ## Adjuntar IGW a la VPC
  VPCGatewayAttachment:
    Type: AWS::EC2::VPCGatewayAttachment
    Properties:
      VpcId: !Ref VPC
      InternetGatewayId: !Ref InternetGateway
##Grupo de seguridad asociado a la instancia
  FlaskSecurityGroup:
    Type: AWS::EC2::SecurityGroup
    Properties:
      GroupDescription: Enable TCP ingress for Flask
      VpcId: !Ref VPC
      SecurityGroupIngress:
        - IpProtocol: tcp
          FromPort: 5000
          ToPort: 5000
          CidrIp: 0.0.0.0/0
        - IpProtocol: tcp
          FromPort: 22
          ToPort: 22
          CidrIp: 0.0.0.0/0
      Tags:
        - Key: Name
          Value: Flask Security Group


  ## Security Group del Balanceador
  BalanceadorSecurityGroup:
    Type: AWS::EC2::SecurityGroup
    Properties:
```

```
      GroupName: BalanceadorSecurityGroup
      GroupDescription: "Acceso HTTP publico para balanceador"
      VpcId: !Ref VPC
      SecurityGroupIngress:
        - IpProtocol: tcp
          FromPort: 80
          ToPort: 80
          CidrIp: 0.0.0.0/0
      Tags:
        - Key: Name
          Value: BalanceadorSecurityGroup

  ## Security Group de las tareas ECS (puerto 5000 desde el balanceador)
  TaskSecurityGroup:
    Type: AWS::EC2::SecurityGroup
    DependsOn: BalanceadorSecurityGroup
    Properties:
      GroupName: TaskSecurityGroup
      GroupDescription: "Permite trafico al puerto Flask 5000 desde el balanceador"
      VpcId: !Ref VPC
      SecurityGroupIngress:
        - IpProtocol: tcp
          FromPort: 5000
          ToPort: 5000
          SourceSecurityGroupId: !Ref BalanceadorSecurityGroup
      Tags:
        - Key: Name
          Value: TaskSecurityGroup

  ## Instancia EC2 de pruebas
  DockerInstance:
    Type: AWS::EC2::Instance
    Properties:
      InstanceType: t2.micro
      ImageId: ami-08a0d1e16fc3f61ea
      KeyName: proy-keypair
      IamInstanceProfile: LabInstanceProfile
      SubnetId: !Ref PublicSubnet1
      SecurityGroupIds:
        - !Ref FlaskSecurityGroup
      Tags:
        - Key: Name
          Value: DockerInstance
      UserData:
        Fn::Base64: !Sub |
```

```
          #!/bin/bash
          dnf update -y
          dnf install -y docker
          systemctl enable docker
          systemctl start docker
  DiskVolume:
    Type: AWS::EC2::Volume
    Properties:
      Size: 100
      AvailabilityZone: !GetAtt DockerInstance.AvailabilityZone
    DeletionPolicy: Snapshot

  DiskMountPoint:
    Type: AWS::EC2::VolumeAttachment
    Properties:
      InstanceId: !Ref DockerInstance
      VolumeId: !Ref DiskVolume
      Device: /dev/sdh

Outputs:

  VPCId:
    Description: ID de la VPC creada
    Value: !Ref VPC

  PublicSubnet1Id:
    Description: "Subred publica 1"
    Value: !Ref PublicSubnet1

  PublicSubnet2Id:
    Description: "Subred pública 2"
    Value: !Ref PublicSubnet2

  TestInstancePublicIP:
    Description: "IP publica de la instancia EC2 de pruebas"
    Value: !GetAtt DockerInstance.PublicIp
```

- **Shell scripts de AWS CLI :**

```
#Crear archivos
mkdir flask_docker
nano flask_docker/funcion.py
nano flask_docker/requirements.txt
nano flask_docker/Dockerfile
```

```
#Probar imagen
cd flask_docker
sudo systemctl start docker #inicio docker
sudo docker build -t flask_container . #Construyo una imagen Docker
sudo docker run -p 5000:5000 -d flask_container #Inicio un contenedor Docker a
partir de la imagen flask_container
sudo docker ps
curl "http://127.0.0.1:5000/maxdiff?month=3&year=2017" #Compruebo que toda va bien

##subir imagen a ECR

#Recupere un token de autenticación y autentique su cliente de Docker en el
registro. Use AWS CLI:
aws ecr get-login-password --region us-east-1 | docker login --username AWS --
password-stdin 339713182596.dkr.ecr.us-east-1.amazonaws.com

#Cuando se complete la creación, etiquete la imagen para poder enviarla a este
repositorio:
docker tag flask_container 339713182596.dkr.ecr.us-east-1.amazonaws.com/proy-
repo:flask_container

#comando para enviar esta imagen al repositorio de AWS recién creado
docker push 339713182596.dkr.ecr.us-east-1.amazonaws.com/proy-repo:flask_container
```

- **Función de AWS CLI 'funcion.py'**:

```
from flask import Flask, request, jsonify
import boto3
from decimal import Decimal
from boto3.dynamodb.conditions import Attr

app = Flask(__name__)

tabla_dy = 'proy-tabla'
region = 'us-east-1'

dynamodb = boto3.resource('dynamodb', region_name=region)
table = dynamodb.Table(tabla_dy)


def media(items, field: str):
    """
    Calcula la media de una lista numérica almacenada en el atributo `field`
    (lista de Decimal / float / int).
    """
```

```
    # Convertimos cada valor a float por si vienen como Decimal
    valores = []
    for i in items:
        valores.append(float(i[field]))

    media_global = sum(valores) / len(valores)
    return media_global

@app.route('/health', methods=['GET'])
def health_check():
    return jsonify({'status': 'healthy'}), 200


@app.route('/temp', methods=['GET'])
def get_temp():
    y, m = request.args.get('year'), request.args.get('month')
    mes_anyo = f"{m}-{y}"
    response = table.scan(
        FilterExpression=Attr("mes_anyo").eq(mes_anyo)
    )

    items = response.get('Items',[])
    if not items:
        return jsonify({"error": f"Datos no encontrados para {mes_anyo}"}, 404)

    media_temp = media(items, "media")

    return jsonify({
    f"Media de temperatura del mes {mes_anyo}": media_temp
    })

@app.route('/maxdiff', methods=['GET'])
def get_maxdiff():
    y, m = int(request.args.get('year')), int(request.args.get('month'))
    mes_anyo = f"{m}-{y}"
    response = table.scan(
        FilterExpression=Attr("mes_anyo").eq(mes_anyo)
    )
    items = response.get('Items',[])
    if not items:
        return jsonify({"error": f"Datos no encontrados para {mes_anyo}"}, 404)

    max_act = 0
    for i in items:
        if float(i['media']) > max_act:
```

```
            max_act = float(i['media'])

    if m == 1:
        m_2 = 12
        y_2 = y-1

    else:
        m_2 = m-1
        y_2 = y

    mes_anyo_ant = f"{m_2}-{y_2}"
    response_ant = table.scan(
        FilterExpression=Attr("mes_anyo").eq(mes_anyo_ant)
    )
    items_ant = response_ant.get('Items',[])
    if not items_ant:
        return jsonify({"error": f"Datos no encontrados para mes anterior"}, 404)

    max_ant = 0
    for i in items_ant:
        if float(i['media']) > max_ant:
            max_ant = float(i['media'])

    max_diff = max_act-max_ant
    return jsonify({
        f"Diferencia maxima de temperatura del mes {mes_anyo}": max_diff
    })

@app.route('/sd', methods=['GET'])
def get_sd():
    y, m = request.args.get('year'), request.args.get('month')
    mes_anyo = f"{m}-{y}"
    response = table.scan(
        FilterExpression=Attr("mes_anyo").eq(mes_anyo)
    )
    items = response.get('Items',[])
    if not items:
        return jsonify({"error": f"Datos no encontrados para {mes_anyo}"}, 404)

    max = 0
    for i in items:
        if float(i['desviacion']) > max:
            max = float(i['desviacion'])

    return jsonify({
```

```
    f"Maxima desviacion del mes {mes_anyo}": max
    })

if __name__ == '__main__':
    app.run(host='0.0.0.0', port=5000)
```

- Documentos de AWS CLI ‘requirements.txt’ y ‘Dockerfile’:

```
Flask==2.1.1
boto3==1.24.5
werkzeug==2.1.2
Jinja2==3.1.2
MarkupSafe==2.1.1
itsdangerous==2.1.2
click==8.1.3
```

```
# Usamos la imagen base de Python 3.8 sobre Alpine
FROM python:3.8-alpine

# Establecemos el directorio de trabajo dentro del contenedor
WORKDIR /app

# Copiamos el archivo requirements.txt al contenedor
COPY requirements.txt requirements.txt

# Instalamos las dependencias listadas en el requirements.txt
RUN pip install --no-cache-dir -r requirements.txt

# Copiamos todo el contenido de la carpeta local al contenedor
COPY . .

# Especificamos el comando para ejecutar el script Python metadata.py al inicia>
CMD ["python3", "funcion.py"]
```

Resumen de recursos y servicios desplegados

Identificador de recurso	Nombre de recurso	Tipo de recurso en AWS	Comentario (opcional)

vpc-0842e39f3eb0d667a	proy-vpc	VPC	Red principal
subnet-0d5cc650df98dd98c	Proy-public-subnet-1	Subred	Subred publica
subnet-06aee243cbaf21fdf	Proy-public-subnet-2	Subred	Subred publica
i-06cc1064baccd8e67	DockerInstance	Instancia de EC2	Docker
i-0c8f6658282a5c38a	ECS-instance-proy-cluster	Instancia de EC2	Instancia creada con el servicio
i-04739e1f15e19cb2a	ECS-instance-proy-cluster	Instancia de EC2	Instancia creada con el servicio
sg-0a9d6cb1394d451f5	BalanceadroSecurityGroup	Grupo de seguridad	Grupo de seguridad asociado al balanceador
sg-0e5102c2d3fa77f5b	TaskSecurityGroup	Grupo de seguirdad	Grupo de seguridad asociado a las tareas
sg-09db73553c9d47671	FlaskSecurityGroup	Grupo de Seguridad	Grupo de seguridad asociado al Docker
proy-alb-1715306209.us-east-1.elb.amazonaws.com	Proy-alb	Balanceador de carga	Redistribuye el trabajo
arn:aws:elasticloadbalancing:us-east-1:851725627517:targetgroup/proy-tg-flask/e9ab6f881f6314c1	Proy-tg-flask	Grupo de destino	Redirecciona el trafico
vpce-00744cb23b647b8db	Proy-vpc-dynamodb	Punto de conexion	Conecta la vpc con dynamodb
851725627517.dkr.ecr.us-east-1.amazonaws.com/proy-repo	Proy-repo	Repositorio	Almacena imagenes

arn:aws:ecs:us-east-1:851725627517:cluster/proy-cluster	Proy-cluster	Cluster	Grupo de instancias que ejecuta contenedores
	Proy-keypair	Par de claves	
arn:aws:s3:::proy-ingestadatos	Proy-ingestadatos	Bucket de S3	Almacena datos en sucio
arn:aws:lambda:us-east-1:851725627517:function:proy-funcion-procesador-datos	Proy-funcion-procesador-datos	Funcion Lambda	Proceso los datos situados en el proy-ingestadatos y modifica para ponerlos en el formato correcto
arn:aws:lambda:us-east-1:851725627517:function:proy-funcion-notificacion	Proy-funcion-notificacion	Funcion Lambda	Detecta desviación mayor que 0.5
arn:aws:sns:us-east-1:851725627517:proy-alarma-temperatura	Proy-alarma-temperatura	Tema SNS	Envia mensajes a subscriptores
arn:aws:ecs:us-east-1:851725627517:task-definition/proy-task:2	Proy-task	Tarea	Encapsula lo necesario para ejecutar contenedor
arn:aws:cloudformation:us-east-1:851725627517:stack/proy-pila/d44dae80-d758-11f0-a270-12c75d78730d	Proy-pila	Pila de CloudFormation	Lanza infraestructura basica
arn:aws:cloudformation:us-east-1:851725627517:stack/ECS-Console-V2-Service-proy-ecs-service-proy-cluster-e4703a46/0c8e97c0-d75c-11f0-87d2-0affe5514849	ECS-Console-V2-Service-proy-ecs-service-proy-cluster-e4703a46	Pila de CloudFormation	Lanza servicio

rtb-032cc2a96131b2cd8	-	Tabla de enrutamiento	Asocia las dos subredes a la VPC
arn:aws:dynamodb:us-east-1:851725627517:table/proy-tabla	Proy-tabla	Tabla DynamoDB	Tabla de DynamoDB donde guardamos información
arn:aws:ecs:us-east-1:851725627517:service/proy-cluster/proy-ecs-service	proy-ecs-service	Servicio	Servicio que ejecuta tareas para el balanceador de carga